Here is a **register-programming-oriented view** showing what kinds of tables and parameters a switch driver would configure for each block.

Below is a **vendor-neutral, register-programming-oriented view** of the TSN-capable switch datapath. The **IEEE 802.1Q** family defines the behavior and managed objects, but the actual **register names, bitfields, SRAM layout, and table packing are vendor-specific**. So let us use **silicon-style conceptual register names** that a real switch driver would typically map onto hardware tables. The standards anchor is still the same datapath split: **Ingress Port → Relay → Egress Port**, with forwarding/filtering in the relay and metering, queuing, and transmission selection on egress.

1. The model the driver should use

Think of the driver as programming **five families of state**:

1. **Classification state**
   "How do I recognize this frame or stream?"
2. **Forwarding state**
   "Once recognized, which egress port set does it go to?"
3. **Per-stream enforcement state**
   "Does this stream conform, and is it currently allowed to proceed?"
4. **Per-queue shaping/scheduling state**
   "When may this queue transmit?"
5. **Status/counter state**
   "What actually happened?"

That split matches the standards view very well: stream identification associates a received frame with stream parameters; forwarding/filtering is separate; and shaping/scheduling is an egress function.

---

2. Parser block → what the driver typically configures

The parser is usually not programmed with "stream semantics" directly. Instead, the driver configures **what fields are extracted** and **which protocol layers are enabled for classification**.

Typical conceptual registers

```
PARSER_CTRL
PARSER_L2_EN
PARSER_VLAN_EN
PARSER_L3_EN
PARSER_L4_EN
PARSER_KEYSEL[n]
PARSER_OFFSET_PROFILE[n]
```

What these mean

The driver typically enables extraction of:

- ingress port
- destination and source **Media Access Control (MAC)** address
- **Virtual Local Area Network (VLAN)** identifier and priority
- EtherType
- optionally **Internet Protocol (IP)** and **User Datagram Protocol (UDP) / Transmission Control Protocol (TCP)** fields

because IEEE stream identification can use all of those as matching inputs.

What you would actually set

For a TSN stream-identification capable port, the driver would usually:

- enable VLAN parsing
- enable IP/L4 parsing only if the stream filters need those fields
- choose a key profile for the classifier, for example:
    - `{ingress_port, DA, VID}`
    - `{ingress_port, SA, VID}`
    - `{DA, VID, IP dst, UDP dst}`

Practical insight

This is often where driver writers overconfigure the switch. If your stream rules only need `{DA, VID}`, turning on deep parsing for every port wastes lookup width and power for no benefit.

---

3. Classifier / TCAM / Stream-Identification block

This is the first place where a real TSN-aware switch driver programs **per-stream state**.

IEEE and AUTOSAR both support the idea that incoming frames are matched on protocol fields and associated with a stream object; AUTOSAR's Ethernet Switch Driver also says stream identification is applied at the **ingress side**, and only **passive stream identification** is supported there.

Typical conceptual table

```
CLASS_RULE[n].KEY
CLASS_RULE[n].MASK
CLASS_RULE[n].INGRESS_PORT_BITMAP
CLASS_RULE[n].MATCH_ENABLE
CLASS_RULE[n].STREAM_HANDLE
```

```
CLASS_RULE[n].PSFP_PROFILE_PTR
CLASS_RULE[n].DEFAULT_IPV
CLASS_RULE[n].HIT_COUNTER
```

What a driver programs here

Each rule usually contains:

- **match fields** and masks
- allowed ingress ports
- resulting **stream handle**
- pointer to the **Per-Stream Filtering and Policing (PSFP)** profile
- possibly a default internal priority value

IEEE stream-identification work explicitly ties classification to later actions such as flow meters, gates, packet-length filters, and queue assignment.

Example rule

```
Rule 12:
  match ingress_port = 2
  match DA = 91:E0:F0:00:10:01
  match VID = 100
  -> stream_handle = 12
  -> psfp_profile = 6
  -> default_ipv = 4
```

AUTOSAR mapping

If you are using AUTOSAR **EthSwt**, this is the kind of state represented by:

- `EthSwtStreamIdentification`
- stream filter subcontainers for MAC/IP/TCP/UDP matching criteria.

---

4. Ingress sanity / ingress policer block

This is not the same as PSFP. This is the coarse early-defense block for:

- basic ingress filtering
- coarse rate limiting
- malformed traffic rejection
- possibly port-based policers

AUTOSAR's Ethernet Switch Driver release history explicitly mentions **configuration of rate policers on ingress side**.

Typical conceptual registers

```
ING_PORT_CTRL[p]
ING_POLICER[p][k].RATE
ING_POLICER[p][k].BURST
ING_POLICER[p][k].ACTION
ING_DROP_STATUS[p]
ING_ERR_COUNTER[p]
```

What the driver programs

Per ingress port or per coarse ingress class:

- rate
- burst
- drop/mark action
- possibly storm-control style limits

Why keep this separate from PSFP

Use ingress policers for:

- noisy or untrusted edge ports
- broad traffic containment

Use PSFP for:

- **identified streams** with TSN semantics

---

5. VLAN / FDB / Relay engine

This is the forwarding core. IEEE 802.1Q bridging forwards or filters MAC frames based on **address and VLAN information**, and the relay portion is distinct from stream identification.

Typical conceptual tables

```
PORT_PVID[p]
VLAN_MEMBER[vid]
VLAN_TAG_CTRL[vid][p]
FDB_KEY = {DA, VID}
FDB_DATA = {EGRESS_PORT_BITMAP, STATIC, AGEABLE}
MDB_KEY = {GROUP_DA, VID}
MDB_DATA = {EGRESS_PORT_BITMAP}
```

What the driver programs

At minimum:

- **Port VLAN Identifier (PVID)** for untagged ingress handling
- VLAN membership bitmap per VLAN
- tagged vs untagged egress behavior per port/VLAN

- **Forwarding Database (FDB)** entries for unicast `{DA, VID}`
- multicast database entries for group addresses

Example

```
PORT_PVID[2] = 100

VLAN_MEMBER[100] = ports {2,7,9}
VLAN_TAG_CTRL[100][2] = ingress accept tagged only
VLAN_TAG_CTRL[100][7] = egress tagged
VLAN_TAG_CTRL[100][9] = egress tagged

FDB[{91:E0:F0:00:10:01, 100}] = port {7}
```

Important

The stream handle does **not** replace the FDB. The classifier decides “what stream is this?”, while the relay decides “where does it go?”

---

6. PSFP / Qci block

This is where the switch consumes the stream handle and turns it into **actual per-stream enforcement**.

IEEE material states that stream identification associates each received frame with parameters that can identify:

- an SDU size filter
- a stream gate
- a flow meter

for subsequent actions.

A good driver model is to split PSFP into three linked tables.

6.1 Stream Filter Instance Table

Typical conceptual table

```
PSFP_FILTER[f].STREAM_HANDLE
PSFP_FILTER[f].MAX_SDU
PSFP_FILTER[f].GATE_ID
PSFP_FILTER[f].METER_ID
PSFP_FILTER[f].IPV_OVERRIDE
PSFP_FILTER[f].MATCH_PRIORITY
PSFP_FILTER[f].COUNTER_PTR
```

What the driver programs

For each identified stream:

- maximum frame size
- which stream gate instance applies
- which flow meter applies
- optional internal priority value override

This is the closest hardware analogue to the IEEE “stream filter instance table.”

6.2 Stream Gate Table

The stream gate is **per stream**, not per queue.

Typical conceptual registers

```
SGATE[g].ADMIN_GATE_STATE
SGATE[g].OPER_GATE_STATE
SGATE[g].ADMIN_IPV
SGATE[g].OPER_IPV
SGATE[g].BASE_TIME_SEC
SGATE[g].BASE_TIME_NSEC
SGATE[g].CYCLE_TIME_NUM
SGATE[g].CYCLE_TIME_DEN
SGATE[g].CYCLE_TIME_EXT
SGATE[g].ADMIN_CTRL_LIST_LEN
SGATE[g].CTRL_LIST[i].GATE_STATE
SGATE[g].CTRL_LIST[i].TIME_INTERVAL
SGATE[g].CONFIG_CHANGE
SGATE[g].CONFIG_CHANGE_TIME
```

What these are based on

IEEE public YANG work for PSFP and scheduled traffic exposes these kinds of administrative and operational objects:

- gate states
- control list length
- control-list entries
- cycle time
- base time
- config-change signaling.

Example

```
SGATE[3].BASE_TIME = 12 s + 500000000 ns
SGATE[3].CYCLE_TIME = 1 / 1000 s   ; 1 ms
SGATE[3].CTRL_LIST[0] = {gate=open,  interval=100000 ns}
SGATE[3].CTRL_LIST[1] = {gate=close, interval=900000 ns}
SGATE[3].CONFIG_CHANGE = 1
```

That means the stream is only admitted for the first 100 microseconds of each 1 millisecond cycle.

AUTOSAR mapping

This is directly analogous to `EthSwtStreamGate`, including `EthSwtStreamGateIPV`.

6.3 Flow Meter Table

IEEE PSFP flow meters use a bandwidth-profile style parameter set. Public IEEE material lists these per-meter parameters:

- **Committed Information Rate (CIR)**
- **Committed Burst Size (CBS)**
- **Excess Information Rate (EIR)**
- **Excess Burst Size (EBS)**
- Coupling Flag
- Color Mode
- DropOnYellow
- MarkAllFramesRedEnable
- MarkAllFramesRed.

Typical conceptual registers

```
FMETER[m].CIR
FMETER[m].CBS
FMETER[m].EIR
FMETER[m].EBS
FMETER[m].COUPLING_FLAG
FMETER[m].COLOR_MODE
FMETER[m].DROP_ON_YELLOW
FMETER[m].MARK_ALL_RED_EN
FMETER[m].MARK_ALL_RED
FMETER[m].GREEN_CNT
FMETER[m].YELLOW_CNT
FMETER[m].RED_CNT
```

Example

```
FMETER[5].CIR  = 50000000   ; 50 Mbit/s
FMETER[5].CBS  = 32000      ; bytes
FMETER[5].EIR  = 0
FMETER[5].EBS  = 0
FMETER[5].COUPLING_FLAG = 0
FMETER[5].COLOR_MODE    = color_blind
FMETER[5].DROP_ON_YELLOW = 1
```

AUTOSAR mapping

This corresponds closely to `EthSwtFlowMetering`, including `EthSwtFlowMeterCF` for the coupling flag.

---

7. Egress queue manager

After PSFP admits the frame, the switch has to place it into an egress queue.

Typical conceptual registers

```
EGR_QMAP[p][ipv] = queue_id
EGR_Q_LIMIT[p][q]
EGR_Q_DROP_MODE[p][q]
EGR_Q_COUNTER[p][q]
QUEUE_MAX_SDU[p][q]
```

What the driver programs

Per egress port and queue:

- internal-priority-to-queue mapping
- queue depth or threshold
- drop mode
- optional per-queue maximum SDU

Why `QUEUE_MAX_SDU` matters

For scheduled traffic, IEEE material notes that if a gate-open interval is too short for a queued frame to finish before gate close, the frame may never be transmitted and can eventually be discarded; a queue length limit such as `queueMaxSDU` helps prevent such impossible-to-send frames from blocking useful operation.

Example

```
EGR_QMAP[7][4] = 4
EGR_Q_LIMIT[7][4] = 64 descriptors
QUEUE_MAX_SDU[7][4] = 1600
```

---

8. Time-Aware Shaper (TAS, IEEE 802.1Qbv) block

This is the **per-queue gate scheduler** on egress.

IEEE public models expose these core managed objects for a port:

- admin and operational gate states

- admin and operational control list
- base time
- cycle time
- cycle time extension
- config-change.

Typical conceptual registers

```
TAS[p].GATE_ENABLE
TAS[p].ADMIN_GATE_STATES
TAS[p].OPER_GATE_STATES
TAS[p].ADMIN_CTRL_LIST_LEN
TAS[p].CTRL_LIST[i].GATE_MASK
TAS[p].CTRL_LIST[i].TIME_INTERVAL_NS
TAS[p].BASE_TIME_SEC
TAS[p].BASE_TIME_NSEC
TAS[p].CYCLE_TIME_NUM
TAS[p].CYCLE_TIME_DEN
TAS[p].CYCLE_TIME_EXT_NS
TAS[p].CONFIG_CHANGE
TAS[p].CONFIG_CHANGE_TIME
```

What the driver programs

For each egress port:

- whether TAS is enabled
- initial gate states
- gate control list length
- each gate control entry:
  - gate mask
  - interval
- base time in the synchronized **Precision Time Protocol (PTP)** / **generalized Precision Time Protocol (gPTP)** timescale
- cycle time
- config-change handshake

Gate mask meaning

IEEE public text describes `gateStatesValue` as an 8-bit field where each bit corresponds to a traffic class, with bit value 1 meaning open and 0 meaning closed.

Example

```
TAS[7].ADMIN_GATE_STATES = 0x00
TAS[7].BASE_TIME = 15 s + 0 ns
TAS[7].CYCLE_TIME = 1 / 1000 s   ; 1 ms
TAS[7].CTRL_LIST[0] = {GATE_MASK = 0x10, TIME_INTERVAL_NS = 120000}
TAS[7].CTRL_LIST[1] = {GATE_MASK = 0x01, TIME_INTERVAL_NS = 880000}
TAS[7].CONFIG_CHANGE = 1
```

Here queue 4 is open for 120 microseconds each millisecond, and queue 0 is open for the rest.

9. Credit-Based Shaper (CBS, IEEE 802.1Qav) block

CBS is per **traffic class queue**, not per stream.

AUTOSAR's TSN explanation says CBS is an egress-side functional block that makes frames available for transmission based on a credit value; if credit is zero or positive, the queue is eligible, and if negative, it is not. It also states that `idleSlope` is the main configuration parameter, and gives `highCredit` and `lowCredit` as useful bounds. It further notes that `sendSlope = idleSlope - portTransmitRate`.

Typical conceptual registers

```
CBS[p][q].ENABLE
CBS[p][q].IDLE_SLOPE
CBS[p][q].SEND_SLOPE
CBS[p][q].HI_CREDIT
CBS[p][q].LO_CREDIT
CBS[p][q].CUR_CREDIT
CBS[p][q].STATS
```

What the driver programs

Per egress queue:

- enable/disable CBS
- `idleSlope`
- either `sendSlope` explicitly or let hardware derive it from port rate
- high and low credit bounds if programmable
- optionally reset current credit on reconfiguration

Example

```
CBS[7][4].ENABLE      = 1
CBS[7][4].IDLE_SLOPE  = 55000000   ; 55 Mbit/s
CBS[7][4].SEND_SLOPE  = 55000000 - 1000000000  ; if 1 Gbit/s port
CBS[7][4].HI_CREDIT   = 12000
CBS[7][4].LO_CREDIT   = -16000
```

AUTOSAR mapping

These correspond to:

- `EthSwtPortEgressQueueCreditBasedShaperIdleSlope`
- `...UpperBoundary`
- `...LowerBoundary`
  in the AUTOSAR explanation and switch-driver abstractions.

10. MAC launch arbiter / egress scheduler

This is the last selection point before transmission.

AUTOSAR requirements say that, if supported by the hardware, the switch driver shall support configuration of:

- the **transmission selection algorithm** per egress queue, such as CBS, Asynchronous Traffic Shaping, Enhanced Transmission Selection, or unshaped
- and the **scheduling algorithm** per egress port, such as strict priority, weighted round robin, or deficit round robin.

Typical conceptual registers

```
TX_ARB[p].PORT_SCHED_MODE
TX_ARB[p].STRICT_PRIO_ORDER
TX_ARB[p].WRR_WEIGHT[q]
TX_ARB[p].DRR_QUANTUM[q]
TX_ARB[p].PREEMPT_CLASS_MASK
TX_ARB[p].TX_SEL_ALGO[q]
```

What the driver programs

Per port:

- global egress scheduler mode
- strict-priority ordering or weights/quantums
- queue transmission-selection algorithm:
    - unshaped
    - CBS
    - ATS
    - ETS

Example

```
TX_ARB[7].PORT_SCHED_MODE = STRICT_PRIORITY
TX_ARB[7].TX_SEL_ALGO[4]  = CBS
TX_ARB[7].TX_SEL_ALGO[0]  = UNSHAPED
```

That means queue 4 is governed by CBS eligibility and then participates in strict-priority launch among the eligible queues.

11. What “reservation state” looks like in registers

This is the key conversion from the control plane into hardware.

A reservation protocol or centralized network controller does **not** inject a magical “reserved” bit into each packet. Instead, it installs these concrete hardware states:

```
stream reservation
   → CLASS_RULE[n]
   → PSFP_FILTER[f]
   → SGATE[g]
   → FMETER[m]
   → EGR_QMAP[p][ipv]
   → TAS[p].CTRL_LIST[*]
   → CBS[p][q]
   → VLAN_MEMBER[vid], FDB entries
```

That mapping is exactly what IEEE 802.1Qcc-style centralized configuration means when it says the central entity can configure traffic classes, resource-allocation classes, VLANs, and transmission-gate schemes in the bridges.

---

12. A good driver initialization order

A practical switch driver should not program these blocks randomly.

Recommended order

Step 1: Port and parser baseline

- port enable defaults
- parser key extraction
- PVID / tag handling defaults

Step 2: VLAN and forwarding baseline

- VLAN membership
- FDB / multicast entries
- default flooding policy

Step 3: Stream identification

- classifier/TCAM rules
- stream-handle bindings

Step 4: PSFP

- stream filter entries
- stream gates
- flow meters

Step 5: Queue mapping

- internal-priority-to-queue map
- queue limits
- queueMaxSDU if relevant

Step 6: Egress shaping

- TAS base time / cycle / GCL
- CBS parameters

Step 7: Launch arbitration

- per-port scheduler mode
- per-queue transmission-selection algorithm

Step 8: Commit

- set `CONFIG_CHANGE`-style controls for TAS / gates
- clear counters
- enable traffic

This order reduces the risk of frames entering partially configured hardware.

---

13. A concrete worked example

Suppose you want one front-camera stream on port 2 to go out port 7 with:

- VLAN 100
- stream-specific PSFP
- queue 4 on egress
- TAS window of 120 microseconds every 1 millisecond
- CBS shaping around 55 Mbit/s

A conceptual driver image would look like this:

```
PORT_PVID[2] = 100
VLAN_MEMBER[100] = {2,7}
FDB[{DA_cam,100}] = {7}

CLASS_RULE[12]:
  KEY  = {ingress_port=2, DA=DA_cam, VID=100}
  MASK = exact
  STREAM_HANDLE = 12
  PSFP_PROFILE_PTR = 6
  DEFAULT_IPV = 4

PSFP_FILTER[6]:
  STREAM_HANDLE = 12
```

```
  MAX_SDU = 1600
  GATE_ID = 3
  METER_ID = 5
  IPV_OVERRIDE = 4

SGATE[3]:
  BASE_TIME = 20 s + 0 ns
  CYCLE_TIME = 1/1000 s
  CTRL_LIST[0] = {open, 120000 ns}
  CTRL_LIST[1] = {close, 880000 ns}
  CONFIG_CHANGE = 1

FMETER[5]:
  CIR = 50000000
  CBS = 32000
  EIR = 0
  EBS = 0
  DROP_ON_YELLOW = 1

EGR_QMAP[7][4] = 4
QUEUE_MAX_SDU[7][4] = 1600

TAS[7]:
  CTRL_LIST[0] = {GATE_MASK for queue4 open, 120000 ns}
  CTRL_LIST[1] = {best-effort queues open, 880000 ns}
  BASE_TIME = 20 s + 0 ns
  CYCLE_TIME = 1/1000 s
  CONFIG_CHANGE = 1

CBS[7][4]:
  ENABLE = 1
  IDLE_SLOPE = 55000000
  HI_CREDIT = 12000
  LO_CREDIT = -16000

TX_ARB[7]:
  PORT_SCHED_MODE = STRICT_PRIORITY
  TX_SEL_ALGO[4] = CBS
  TX_SEL_ALGO[0] = UNSHAPED
```

This is not a real vendor register map, but it is very close to how a real driver has to think.

---

14. If you are using AUTOSAR EthSwt

The nearest AUTOSAR configuration objects for this view are:

- `EthSwtStreamIdentification`
- `EthSwtPSFP`
- `EthSwtStreamGate`
- `EthSwtFlowMetering`
- per-egress-queue transmission-selection algorithm
- ingress rate policers.

So the AUTOSAR switch driver abstraction is already pointing in exactly this direction, even though the final hardware register names are silicon-specific.

---

15. The one caution that saves pain later

Do not configure TAS and CBS independently and assume they will compose nicely. IEEE discussion material notes that when CBS and TAS are combined, the TAS gate may decline a frame if the transmission cannot finish before gate close; and IEEE material also warns that frames longer than what a queue's gate windows can support may become impossible to transmit unless queue length/SDU limits are configured sensibly.

That means in driver terms:

- size limits
- TAS windows
- queue mapping
- and CBS rate
  must be configured as one consistent set.

If you want, I can next turn this into a **pseudo-C driver skeleton** with initialization functions like `program_vlan_table()`, `program_stream_filter()`, `program_psfp_gate()`, `program_cbs()`, and `commit_tas_schedule()`.

Below is a **vendor-neutral pseudo-C driver skeleton** for a **Time-Sensitive Networking (TSN)** capable automotive Ethernet switch.

The register names are **illustrative**, not tied to a specific chipset. The goal is to show how a switch driver would be structured around:

- **Virtual Local Area Network (VLAN) / Forwarding Database (FDB)** programming
- stream identification
- **Per-Stream Filtering and Policing (PSFP)**
- **Time-Aware Shaper (TAS)**
- **Credit-Based Shaper (CBS)**
- queue mapping and egress scheduling

---

1. High-level shape

```
/*
 * Vendor-neutral pseudo-C skeleton for a TSN-capable Ethernet switch
driver.
 *
 * Notation:
 *   - MMIO register names are conceptual
 *   - Table indices are logical, not real silicon offsets
```

```
 *   - All time values are assumed to be already converted to hardware
units
 */

#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>

/* ========================
 * Basic helpers
 * ========================= */

typedef volatile uint8_t  vuint8_t;
typedef volatile uint16_t vuint16_t;
typedef volatile uint32_t vuint32_t;
typedef volatile uint64_t vuint64_t;

struct switch_dev {
    uintptr_t base;
    uint32_t num_ports;
    uint32_t num_queues_per_port;
};

/* MMIO helpers */
static inline void writel(struct switch_dev *sw, uint32_t off, uint32_t
val)
{
    *((vuint32_t *)(sw->base + off)) = val;
}

static inline uint32_t readl(struct switch_dev *sw, uint32_t off)
{
    return *((vuint32_t *)(sw->base + off));
}

/* Poll helper: pseudo only */
static bool wait_until_set(struct switch_dev *sw, uint32_t off, uint32_t
mask, uint32_t timeout_loops)
{
    while (timeout_loops--) {
        if (readl(sw, off) & mask)
            return true;
    }
    return false;
}
```

---

2. Conceptual register map

```
/* ========================
 * Conceptual register offsets
 * ========================= */

/* Global / parser */
#define REG_GLOBAL_CTRL                  0x0000
#define REG_PARSER_CTRL                  0x0010

/* Per-port ingress */
#define REG_PORT_CTRL(p)                 (0x1000 + ((p) * 0x100))
#define REG_PORT_PVID(p)                 (0x1004 + ((p) * 0x100))
```

```
#define REG_PORT_ING_POLICER_RATE(p)     (0x1010 + ((p) * 0x100))
#define REG_PORT_ING_POLICER_BURST(p)    (0x1014 + ((p) * 0x100))

/* VLAN / forwarding tables: accessed through indirect windows */
#define REG_VLAN_TBL_CTRL                0x2000
#define REG_VLAN_TBL_KEY                 0x2004
#define REG_VLAN_TBL_DATA0               0x2008
#define REG_VLAN_TBL_DATA1               0x200C

#define REG_FDB_TBL_CTRL                 0x2100
#define REG_FDB_TBL_KEY0                 0x2104
#define REG_FDB_TBL_KEY1                 0x2108
#define REG_FDB_TBL_DATA0                0x210C
#define REG_FDB_TBL_DATA1                0x2110

/* Stream identification / classifier */
#define REG_CLASS_TBL_CTRL               0x3000
#define REG_CLASS_TBL_KEY0               0x3004
#define REG_CLASS_TBL_KEY1               0x3008
#define REG_CLASS_TBL_KEY2               0x300C
#define REG_CLASS_TBL_MASK0              0x3010
#define REG_CLASS_TBL_MASK1              0x3014
#define REG_CLASS_TBL_MASK2              0x3018
#define REG_CLASS_TBL_ACTION0            0x301C
#define REG_CLASS_TBL_ACTION1            0x3020

/* PSFP: stream filter */
#define REG_PSFP_FILTER_CTRL             0x4000
#define REG_PSFP_FILTER_KEY              0x4004
#define REG_PSFP_FILTER_DATA0            0x4008
#define REG_PSFP_FILTER_DATA1            0x400C

/* PSFP: stream gate */
#define REG_SGATE_CTRL                   0x4100
#define REG_SGATE_KEY                    0x4104
#define REG_SGATE_DATA0                  0x4108
#define REG_SGATE_DATA1                  0x410C
#define REG_SGATE_DATA2                  0x4110
#define REG_SGATE_GCL_CTRL               0x4114
#define REG_SGATE_GCL_DATA0              0x4118
#define REG_SGATE_GCL_DATA1              0x411C

/* Flow meter */
#define REG_FMETER_CTRL                  0x4200
#define REG_FMETER_KEY                   0x4204
#define REG_FMETER_DATA0                 0x4208
#define REG_FMETER_DATA1                 0x420C
#define REG_FMETER_DATA2                 0x4210

/* Queue mapping / queue limits */
#define REG_QMAP_CTRL                    0x5000
#define REG_QMAP_KEY                     0x5004
#define REG_QMAP_DATA                    0x5008

#define REG_QCFG_CTRL                    0x5100
#define REG_QCFG_KEY                     0x5104
#define REG_QCFG_DATA0                   0x5108
#define REG_QCFG_DATA1                   0x510C

/* TAS: per-port schedule */
#define REG_TAS_CTRL(p)                  (0x6000 + ((p) * 0x80))
```

```
#define REG_TAS_BASE_TIME_SEC(p)          (0x6004 + ((p) * 0x80))
#define REG_TAS_BASE_TIME_NSEC(p)         (0x6008 + ((p) * 0x80))
#define REG_TAS_CYCLE_TIME_NUM(p)         (0x600C + ((p) * 0x80))
#define REG_TAS_CYCLE_TIME_DEN(p)         (0x6010 + ((p) * 0x80))
#define REG_TAS_GCL_CTRL(p)               (0x6014 + ((p) * 0x80))
#define REG_TAS_GCL_DATA0(p)              (0x6018 + ((p) * 0x80))
#define REG_TAS_GCL_DATA1(p)              (0x601C + ((p) * 0x80))
#define REG_TAS_CFG_CHANGE(p)             (0x6020 + ((p) * 0x80))
#define REG_TAS_STATUS(p)                 (0x6024 + ((p) * 0x80))

/* CBS: per-port, per-queue */
#define REG_CBS_CTRL(p, q)                (0x7000 + ((p) * 0x100) + ((q) *
0x10))
#define REG_CBS_IDLE_SLOPE(p, q)          (0x7004 + ((p) * 0x100) + ((q) *
0x10))
#define REG_CBS_SEND_SLOPE(p, q)          (0x7008 + ((p) * 0x100) + ((q) *
0x10))
#define REG_CBS_HI_CREDIT(p, q)           (0x700C + ((p) * 0x100) + ((q) *
0x10))
#define REG_CBS_LO_CREDIT(p, q)           (0x7010 + ((p) * 0x100) + ((q) *
0x10))

/* TX arbiter */
#define REG_TX_ARB_CTRL(p)                (0x8000 + ((p) * 0x40))
#define REG_TX_ARB_QUEUE_MODE(p, q)       (0x8004 + ((p) * 0x40) + ((q) *
0x4))

/* Indirect op bits */
#define TBL_OP_WRITE                      (1U << 0)
#define TBL_OP_BUSY                       (1U << 31)

/* TAS bits */
#define TAS_CTRL_ENABLE                   (1U << 0)
#define TAS_CFG_CHANGE_REQ                (1U << 0)
#define TAS_CFG_CHANGE_DONE               (1U << 0)

/* CBS bits */
#define CBS_CTRL_ENABLE                   (1U << 0)
```

---

3. Configuration objects

```
/* ========================
 * Driver-facing config objects
 * ========================= */

struct vlan_entry {
    uint16_t vid;
    uint32_t member_ports_bitmap;
    uint32_t tagged_ports_bitmap;
};

struct fdb_entry {
    uint8_t  mac[6];
    uint16_t vid;
    uint32_t egress_ports_bitmap;
    bool     is_static;
};

struct stream_match {
```

```
    uint32_t ingress_ports_bitmap;
    uint8_t  dst_mac[6];
    uint8_t  src_mac[6];
    uint16_t vid;
    uint8_t  pcp;
    uint16_t ether_type;
    uint32_t ipv4_dst;      /* optional */
    uint16_t udp_dst_port;  /* optional */
};

struct stream_filter_cfg {
    uint32_t stream_handle;
    uint32_t psfp_profile_id;
    uint16_t max_sdu;
    uint16_t gate_id;
    uint16_t meter_id;
    uint8_t  ipv_override;
};

struct stream_gate_entry {
    uint8_t  gate_open;      /* 1 = open, 0 = closed */
    uint32_t interval_ns;
};

#define MAX_SGATE_ENTRIES 16

struct stream_gate_cfg {
    uint16_t gate_id;
    uint8_t  admin_gate_state;
    uint8_t  admin_ipv;
    uint32_t base_time_sec;
    uint32_t base_time_nsec;
    uint32_t cycle_time_num;
    uint32_t cycle_time_den;
    uint32_t num_entries;
    struct stream_gate_entry entries[MAX_SGATE_ENTRIES];
};

struct flow_meter_cfg {
    uint16_t meter_id;
    uint32_t cir;            /* committed information rate */
    uint32_t cbs;            /* committed burst size */
    uint32_t eir;            /* excess information rate */
    uint32_t ebs;            /* excess burst size */
    bool coupling_flag;
    bool drop_on_yellow;
};

struct qmap_cfg {
    uint8_t  port;
    uint8_t  ipv;
    uint8_t  queue_id;
};

struct queue_cfg {
    uint8_t  port;
    uint8_t  queue_id;
    uint16_t queue_max_sdu;
    uint16_t queue_limit;
};
```

```
struct tas_entry {
    uint8_t  gate_mask;      /* one bit per traffic class queue */
    uint32_t interval_ns;
};

#define MAX_TAS_ENTRIES 32

struct tas_schedule_cfg {
    uint8_t  port;
    uint32_t base_time_sec;
    uint32_t base_time_nsec;
    uint32_t cycle_time_num;
    uint32_t cycle_time_den;
    uint32_t num_entries;
    struct tas_entry entries[MAX_TAS_ENTRIES];
};

struct cbs_cfg {
    uint8_t  port;
    uint8_t  queue_id;
    uint32_t idle_slope;
    int32_t  send_slope;
    int32_t  hi_credit;
    int32_t  lo_credit;
};

enum tx_queue_mode {
    TXQ_MODE_UNSHAPED = 0,
    TXQ_MODE_CBS      = 1,
    TXQ_MODE_ETS      = 2,
    TXQ_MODE_STRICT   = 3,
};

struct tx_arb_cfg {
    uint8_t port;
    uint32_t port_sched_mode;
    enum tx_queue_mode queue_mode[8];
};
```

---

4. Common indirect-table helper

```
/* ========================
 * Common indirect table write helper
 * ========================= */

static int table_write(struct switch_dev *sw,
                       uint32_t ctrl_reg,
                       uint32_t key_regs[], size_t num_keys,
                       uint32_t data_regs[], size_t num_data,
                       const uint32_t *key_vals,
                       const uint32_t *data_vals)
{
    size_t i;

    for (i = 0; i < num_keys; i++)
        writel(sw, key_regs[i], key_vals[i]);

    for (i = 0; i < num_data; i++)
        writel(sw, data_regs[i], data_vals[i]);
```

```
    writel(sw, ctrl_reg, TBL_OP_WRITE);

    if (!wait_until_set(sw, ctrl_reg, 0 /* pseudo busy clear */, 100000))
        return -1;

    return 0;
}
```

---

5. VLAN and forwarding

```
/* =========================
 * VLAN / FDB programming
 * ========================= */

int program_vlan_table(struct switch_dev *sw, const struct vlan_entry *cfg)
{
    uint32_t keys[]  = { REG_VLAN_TBL_KEY };
    uint32_t datas[] = { REG_VLAN_TBL_DATA0, REG_VLAN_TBL_DATA1 };
    uint32_t key_vals[1];
    uint32_t data_vals[2];

    key_vals[0]  = cfg->vid;
    data_vals[0] = cfg->member_ports_bitmap;
    data_vals[1] = cfg->tagged_ports_bitmap;

    return table_write(sw, REG_VLAN_TBL_CTRL,
                       keys, 1, datas, 2,
                       key_vals, data_vals);
}

static void pack_mac_key(const uint8_t mac[6], uint32_t *lo, uint32_t *hi)
{
    *lo = ((uint32_t)mac[2] << 24) |
          ((uint32_t)mac[3] << 16) |
          ((uint32_t)mac[4] <<  8) |
          ((uint32_t)mac[5] <<  0);

    *hi = ((uint32_t)mac[0] << 8) |
          ((uint32_t)mac[1] << 0);
}

int program_fdb_entry(struct switch_dev *sw, const struct fdb_entry *cfg)
{
    uint32_t keys[]  = { REG_FDB_TBL_KEY0, REG_FDB_TBL_KEY1 };
    uint32_t datas[] = { REG_FDB_TBL_DATA0, REG_FDB_TBL_DATA1 };
    uint32_t key_vals[2];
    uint32_t data_vals[2];
    uint32_t mac_lo, mac_hi;

    pack_mac_key(cfg->mac, &mac_lo, &mac_hi);

    key_vals[0] = mac_lo;
    key_vals[1] = (mac_hi << 16) | cfg->vid;

    data_vals[0] = cfg->egress_ports_bitmap;
    data_vals[1] = cfg->is_static ? 1U : 0U;

    return table_write(sw, REG_FDB_TBL_CTRL,
```

```
                        keys, 2, datas, 2,
                        key_vals, data_vals);
}
```

---

6. Stream identification / classifier

```
/* ========================
 * Stream identification
 * ======================== */

static void pack_stream_match_key(const struct stream_match *m,
                                  uint32_t *k0, uint32_t *k1, uint32_t *k2,
                                  uint32_t *mask0, uint32_t *mask1,
uint32_t *mask2)
{
    /* Pseudo packing: purely illustrative */
    uint32_t da_lo, da_hi, sa_lo, sa_hi;
    pack_mac_key(m->dst_mac, &da_lo, &da_hi);
    pack_mac_key(m->src_mac, &sa_lo, &sa_hi);

    *k0 = da_lo;
    *k1 = (da_hi << 16) | m->vid;
    *k2 = ((uint32_t)m->pcp << 24) | ((uint32_t)m->ether_type << 8);

    *mask0 = 0xFFFFFFFFU;
    *mask1 = 0xFFFFFFFFU;
    *mask2 = 0xFFFFFFFFU;

    (void)sa_lo; (void)sa_hi; /* optional in this simplified example */
}

int program_stream_identification(struct switch_dev *sw,
                                  uint16_t rule_index,
                                  const struct stream_match *match,
                                  uint32_t stream_handle,
                                  uint16_t psfp_profile,
                                  uint8_t default_ipv)
{
    uint32_t k0, k1, k2, m0, m1, m2;
    uint32_t keys[]  = {
        REG_CLASS_TBL_KEY0, REG_CLASS_TBL_KEY1, REG_CLASS_TBL_KEY2,
        REG_CLASS_TBL_MASK0, REG_CLASS_TBL_MASK1, REG_CLASS_TBL_MASK2
    };
    uint32_t datas[] = { REG_CLASS_TBL_ACTION0, REG_CLASS_TBL_ACTION1 };
    uint32_t key_vals[6];
    uint32_t data_vals[2];

    pack_stream_match_key(match, &k0, &k1, &k2, &m0, &m1, &m2);

    key_vals[0] = k0;
    key_vals[1] = k1;
    key_vals[2] = k2;
    key_vals[3] = m0;
    key_vals[4] = m1;
    key_vals[5] = m2;

    data_vals[0] = (stream_handle & 0xFFFFU) | ((uint32_t)psfp_profile <<
16);
    data_vals[1] = ((uint32_t)default_ipv << 8) |
```

```
                    (match->ingress_ports_bitmap & 0xFFU);

    (void)rule_index; /* real hardware usually uses this as the table index
*/

    return table_write(sw, REG_CLASS_TBL_CTRL,
                       keys, 6, datas, 2,
                       key_vals, data_vals);
}
```

---

7. PSFP filter, gate, meter

```
/* =========================
 * PSFP stream filter
 * ========================= */

int program_stream_filter(struct switch_dev *sw, const struct
stream_filter_cfg *cfg)
{
    uint32_t keys[]  = { REG_PSFP_FILTER_KEY };
    uint32_t datas[] = { REG_PSFP_FILTER_DATA0, REG_PSFP_FILTER_DATA1 };
    uint32_t key_vals[1];
    uint32_t data_vals[2];

    key_vals[0] = cfg->psfp_profile_id;

    data_vals[0] = (cfg->stream_handle & 0xFFFFU) |
                   ((uint32_t)cfg->max_sdu << 16);

    data_vals[1] = ((uint32_t)cfg->gate_id) |
                   ((uint32_t)cfg->meter_id << 16) |
                   ((uint32_t)cfg->ipv_override << 28);

    return table_write(sw, REG_PSFP_FILTER_CTRL,
                       keys, 1, datas, 2,
                       key_vals, data_vals);
}

/* =========================
 * PSFP stream gate
 * ========================= */

int program_psfp_gate(struct switch_dev *sw, const struct stream_gate_cfg
*cfg)
{
    uint32_t keys[]  = { REG_SGATE_KEY };
    uint32_t datas[] = { REG_SGATE_DATA0, REG_SGATE_DATA1, REG_SGATE_DATA2
};
    uint32_t key_vals[1];
    uint32_t data_vals[3];
    uint32_t i;

    key_vals[0] = cfg->gate_id;

    data_vals[0] = ((uint32_t)cfg->admin_gate_state << 0) |
                   ((uint32_t)cfg->admin_ipv << 8);

    data_vals[1] = cfg->base_time_sec;
    data_vals[2] = cfg->base_time_nsec;
```

```
    if (table_write(sw, REG_SGATE_CTRL,
                    keys, 1, datas, 3,
                    key_vals, data_vals) != 0)
        return -1;

    /* Program cycle time */
    writel(sw, REG_SGATE_DATA0, cfg->cycle_time_num);
    writel(sw, REG_SGATE_DATA1, cfg->cycle_time_den);
    writel(sw, REG_SGATE_CTRL, TBL_OP_WRITE);

    /* Program gate control list */
    for (i = 0; i < cfg->num_entries; i++) {
        writel(sw, REG_SGATE_GCL_DATA0,
               ((uint32_t)cfg->entries[i].gate_open << 0));
        writel(sw, REG_SGATE_GCL_DATA1,
               cfg->entries[i].interval_ns);

        writel(sw, REG_SGATE_GCL_CTRL,
               ((cfg->gate_id & 0xFFFFU) << 16) | (i & 0xFFU) |
TBL_OP_WRITE);
    }

    /* Trigger config change */
    writel(sw, REG_SGATE_CTRL, TBL_OP_WRITE);

    return 0;
}

/* =========================
 * Flow meter
 * ========================= */

int program_flow_meter(struct switch_dev *sw, const struct flow_meter_cfg
*cfg)
{
    uint32_t keys[]  = { REG_FMETER_KEY };
    uint32_t datas[] = { REG_FMETER_DATA0, REG_FMETER_DATA1,
REG_FMETER_DATA2 };
    uint32_t key_vals[1];
    uint32_t data_vals[3];

    key_vals[0] = cfg->meter_id;

    data_vals[0] = cfg->cir;
    data_vals[1] = cfg->cbs;
    data_vals[2] = ((cfg->eir & 0xFFFFU) << 0) |
                   ((cfg->ebs & 0xFFFFU) << 16);

    if (table_write(sw, REG_FMETER_CTRL,
                    keys, 1, datas, 3,
                    key_vals, data_vals) != 0)
        return -1;

    /* Flags would normally go in separate bits/registers */
    writel(sw, REG_FMETER_DATA0,
           (cfg->coupling_flag ? (1U << 0) : 0U) |
           (cfg->drop_on_yellow ? (1U << 1) : 0U));
    writel(sw, REG_FMETER_CTRL, TBL_OP_WRITE);

    return 0;
```

```
}
```

---

8. Queue mapping and queue config

```
/* ========================
 * Queue mapping / queue config
 * ========================= */

int program_queue_map(struct switch_dev *sw, const struct qmap_cfg *cfg)
{
    uint32_t keys[]  = { REG_QMAP_KEY };
    uint32_t datas[] = { REG_QMAP_DATA };
    uint32_t key_vals[1];
    uint32_t data_vals[1];

    key_vals[0]  = ((uint32_t)cfg->port << 8) | cfg->ipv;
    data_vals[0] = cfg->queue_id;

    return table_write(sw, REG_QMAP_CTRL,
                       keys, 1, datas, 1,
                       key_vals, data_vals);
}

int program_queue_cfg(struct switch_dev *sw, const struct queue_cfg *cfg)
{
    uint32_t keys[]  = { REG_QCFG_KEY };
    uint32_t datas[] = { REG_QCFG_DATA0, REG_QCFG_DATA1 };
    uint32_t key_vals[1];
    uint32_t data_vals[2];

    key_vals[0] = ((uint32_t)cfg->port << 8) | cfg->queue_id;
    data_vals[0] = cfg->queue_max_sdu;
    data_vals[1] = cfg->queue_limit;

    return table_write(sw, REG_QCFG_CTRL,
                       keys, 1, datas, 2,
                       key_vals, data_vals);
}
```

---

9. TAS schedule

```
/* ========================
 * TAS (Time-Aware Shaper)
 * ========================= */

int commit_tas_schedule(struct switch_dev *sw, const struct
tas_schedule_cfg *cfg)
{
    uint32_t i;

    writel(sw, REG_TAS_BASE_TIME_SEC(cfg->port),  cfg->base_time_sec);
    writel(sw, REG_TAS_BASE_TIME_NSEC(cfg->port), cfg->base_time_nsec);
    writel(sw, REG_TAS_CYCLE_TIME_NUM(cfg->port), cfg->cycle_time_num);
    writel(sw, REG_TAS_CYCLE_TIME_DEN(cfg->port), cfg->cycle_time_den);

    for (i = 0; i < cfg->num_entries; i++) {
```

```
        writel(sw, REG_TAS_GCL_DATA0(cfg->port), cfg-
>entries[i].gate_mask);
        writel(sw, REG_TAS_GCL_DATA1(cfg->port), cfg-
>entries[i].interval_ns);

        /* pseudo: write entry index + commit */
        writel(sw, REG_TAS_GCL_CTRL(cfg->port), (i & 0xFFU) |
TBL_OP_WRITE);
    }

    /* Request atomic schedule activation */
    writel(sw, REG_TAS_CFG_CHANGE(cfg->port), TAS_CFG_CHANGE_REQ);

    if (!wait_until_set(sw, REG_TAS_STATUS(cfg->port), TAS_CFG_CHANGE_DONE,
100000))
        return -1;

    writel(sw, REG_TAS_CTRL(cfg->port), TAS_CTRL_ENABLE);

    return 0;
}
```

---

10. CBS and TX arbitration

```
/* =========================
 * CBS (Credit-Based Shaper)
 * ========================= */

int program_cbs(struct switch_dev *sw, const struct cbs_cfg *cfg)
{
    writel(sw, REG_CBS_IDLE_SLOPE(cfg->port, cfg->queue_id), (uint32_t)cfg-
>idle_slope);
    writel(sw, REG_CBS_SEND_SLOPE(cfg->port, cfg->queue_id), (uint32_t)cfg-
>send_slope);
    writel(sw, REG_CBS_HI_CREDIT(cfg->port, cfg->queue_id),  (uint32_t)cfg-
>hi_credit);
    writel(sw, REG_CBS_LO_CREDIT(cfg->port, cfg->queue_id),  (uint32_t)cfg-
>lo_credit);
    writel(sw, REG_CBS_CTRL(cfg->port, cfg->queue_id), CBS_CTRL_ENABLE);

    return 0;
}

/* =========================
 * TX arbiter / queue mode
 * ========================= */

int program_tx_arbiter(struct switch_dev *sw, const struct tx_arb_cfg *cfg)
{
    uint32_t q;

    writel(sw, REG_TX_ARB_CTRL(cfg->port), cfg->port_sched_mode);

    for (q = 0; q < 8; q++)
        writel(sw, REG_TX_ARB_QUEUE_MODE(cfg->port, q), (uint32_t)cfg-
>queue_mode[q]);

    return 0;
}
```

11. Optional ingress policer

```
/* ========================
 * Ingress port policer
 * ======================== */

int program_ingress_policer(struct switch_dev *sw,
                            uint8_t port,
                            uint32_t rate,
                            uint32_t burst)
{
    writel(sw, REG_PORT_ING_POLICER_RATE(port),  rate);
    writel(sw, REG_PORT_ING_POLICER_BURST(port), burst);
    return 0;
}
```

12. Global bring-up

```
/* ========================
 * Global init
 * ======================== */

int switch_hw_init(struct switch_dev *sw)
{
    /* Enable parser blocks needed for TSN classification */
    writel(sw, REG_GLOBAL_CTRL, 0x1);
    writel(sw, REG_PARSER_CTRL,
           (1U << 0) | /* L2 enable */
           (1U << 1) | /* VLAN enable */
           (1U << 2) | /* L3 enable if needed */
           (1U << 3)); /* L4 enable if needed */

    return 0;
}
```

13. Example end-to-end setup

```
/* ========================
 * Example setup sequence
 * ======================== */

int setup_front_camera_stream(struct switch_dev *sw)
{
    struct vlan_entry vlan100 = {
        .vid = 100,
        .member_ports_bitmap = (1U << 2) | (1U << 7),
        .tagged_ports_bitmap = (1U << 2) | (1U << 7),
    };

    struct fdb_entry fdb = {
        .mac = {0x91, 0xE0, 0xF0, 0x00, 0x10, 0x01},
        .vid = 100,
        .egress_ports_bitmap = (1U << 7),
```

```
    .is_static = true,
};

struct stream_match match = {
    .ingress_ports_bitmap = (1U << 2),
    .dst_mac = {0x91, 0xE0, 0xF0, 0x00, 0x10, 0x01},
    .vid = 100,
    .pcp = 5,
    .ether_type = 0x22F0, /* example */
};

struct stream_filter_cfg filter = {
    .stream_handle   = 12,
    .psfp_profile_id = 6,
    .max_sdu         = 1600,
    .gate_id         = 3,
    .meter_id        = 5,
    .ipv_override    = 4,
};

struct stream_gate_cfg gate = {
    .gate_id = 3,
    .admin_gate_state = 1,
    .admin_ipv = 4,
    .base_time_sec = 20,
    .base_time_nsec = 0,
    .cycle_time_num = 1,
    .cycle_time_den = 1000, /* 1 ms */
    .num_entries = 2,
    .entries = {
        { .gate_open = 1, .interval_ns = 120000 },
        { .gate_open = 0, .interval_ns = 880000 },
    },
};

struct flow_meter_cfg meter = {
    .meter_id = 5,
    .cir = 50000000,   /* 50 Mbit/s */
    .cbs = 32000,
    .eir = 0,
    .ebs = 0,
    .coupling_flag = false,
    .drop_on_yellow = true,
};

struct qmap_cfg qmap = {
    .port = 7,
    .ipv = 4,
    .queue_id = 4,
};

struct queue_cfg qcfg = {
    .port = 7,
    .queue_id = 4,
    .queue_max_sdu = 1600,
    .queue_limit = 64,
};

struct tas_schedule_cfg tas = {
    .port = 7,
    .base_time_sec = 20,
```

```
        .base_time_nsec = 0,
        .cycle_time_num = 1,
        .cycle_time_den = 1000, /* 1 ms */
        .num_entries = 2,
        .entries = {
            { .gate_mask = (1U << 4), .interval_ns = 120000 },
            { .gate_mask = (1U << 0), .interval_ns = 880000 },
        },
    };

    struct cbs_cfg cbs = {
        .port = 7,
        .queue_id = 4,
        .idle_slope = 55000000,
        .send_slope = -945000000, /* for 1 Gbit/s link example */
        .hi_credit = 12000,
        .lo_credit = -16000,
    };

    struct tx_arb_cfg arb = {
        .port = 7,
        .port_sched_mode = 0, /* strict priority example */
        .queue_mode = {
            TXQ_MODE_UNSHAPED,
            TXQ_MODE_UNSHAPED,
            TXQ_MODE_UNSHAPED,
            TXQ_MODE_UNSHAPED,
            TXQ_MODE_CBS,      /* queue 4 */
            TXQ_MODE_UNSHAPED,
            TXQ_MODE_UNSHAPED,
            TXQ_MODE_UNSHAPED,
        },
    };

    if (switch_hw_init(sw) != 0) return -1;
    if (program_vlan_table(sw, &vlan100) != 0) return -1;
    if (program_fdb_entry(sw, &fdb) != 0) return -1;

    if (program_stream_identification(sw, 12, &match, 12, 6, 4) != 0)
return -1;
    if (program_stream_filter(sw, &filter) != 0) return -1;
    if (program_psfp_gate(sw, &gate) != 0) return -1;
    if (program_flow_meter(sw, &meter) != 0) return -1;

    if (program_queue_map(sw, &qmap) != 0) return -1;
    if (program_queue_cfg(sw, &qcfg) != 0) return -1;

    if (program_cbs(sw, &cbs) != 0) return -1;
    if (program_tx_arbiter(sw, &arb) != 0) return -1;
    if (commit_tas_schedule(sw, &tas) != 0) return -1;

    return 0;
}
```

---

14. What this skeleton is showing

This code intentionally mirrors the hardware datapath:

```
Parser
  → Stream identification
  → VLAN/FDB forwarding
  → PSFP filter/gate/meter
  → Queue map / queue config
  → TAS gate
  → CBS
  → TX arbiter
```

And it also reflects the control-plane-to-hardware translation:

```
reservation / design intent
  → classifier rules
  → PSFP profiles
  → TAS schedule
  → CBS parameters
  → queue mapping
  → forwarding state
```

---

15. What you would add in a real driver

A real implementation would still need:

- read-modify-write helpers
- table readback and verification
- counter access
- interrupt handling
- gPTP-synchronized time conversion for `base_time`
- locking around config changes
- validation for impossible TAS windows vs frame size
- error/status decoding
- shadow config + atomic commit